

오늘의집페이 Backend Engineer 지원

박준호 | Java/Spring Boot 백엔드 개발자

junho6667@gmail.com | 010-3525-6275 | voice-link.co.kr | github.com/junho0831

오늘의집페이가 결제 | 정산 | 매입 시스템을 직접 운영하며 금액 정합성과 트랜잭션 안정성을 핵심으로 다룬다는 점에 주목했습니다. 저는 기능 구현에서 끝내지 않고, 운영 중 발생할 수 있는 중복 처리와 상태 불일치, 재처리 실패를 구체적인 제약 조건과 복구 경로로 줄여 온 백엔드 개발자입니다. 공고에 적힌 "실제 배포 후 운영 환경에서도 통하는 품질"이 제가 계속 해결해 온 문제와 가장 가깝습니다.

엔셀 DataForge에서는 RDB 인덱스와 unique constraint로 중복 신청 | 승인을 막고, Redis TTL을 기준으로 Refresh Token의 저장 | 검증 | 회전 | 삭제를 일원화했습니다. Elasticsearch 장애 시에는 DB fallback으로 전환하되 응답과 정렬 정책을 맞춰 서비스가 계속 동작하도록 했습니다. Prism 배치에서는 업로드 성공과 DB commit 이후에만 원본을 삭제하고 unique+upsert를 적용해 반복 실행에서도 데이터 유실과 중복 적재를 방지했습니다. SMIP에서는 공통 예외 | validation 응답과 회귀 테스트를 도입해 장애 분석 리드타임을 40분에서 12분으로 줄이고 동일 오류 재발률을 30% 낮췄습니다.

개인 서비스 VoiceLink에서는 Redis Lua Script로 실시간 후보 선점을 원자화하고, DB Outbox와 FOR UPDATE SKIP LOCKED로 발행 작업을 선점하며, PESSIMISTIC_WRITE로 세션 종료를 직렬화했습니다. 취소 직후 stale 결과, 메시지 유실, 중복 종료처럼 재현이 어려운 동시성 문제를 상태 기준과 복구 경로로 풀어냈습니다. 이 경험은 결제 도메인 자체를 대신하지는 않지만, 멍등성 | 재시도 | 트랜잭션 정합성을 설계하고 설명할 수 있는 기반입니다.

AI는 구현 | 디버깅 | 테스트 | 문서화 속도를 높이는 협업 도구로 사용합니다. 다만 생성 결과를 그대로 적용하지 않고, 아키텍처와 transaction 경계, lock 전략, 운영 실패 시나리오는 직접 검토하고 결정합니다. 오늘의집페이에서도 결제 | 정산 시스템을 빠르게 이해하면서 운영팀이 확인하고 복구할 수 있는 백오피스와 재발 방지 구조를 만드는 데 기여하겠습니다.

박준호 드림